

Université Laval

IFT-2007 Analyse et conception des systèmes orientés objets
GLO-2004 Génie logiciel orienté objet



Projet de session : BâtiBloc

Réalisé par: Équipe 05

Nom	NI
Youssef Dhaoui	537 285 159
Abdallah Chenah	537 086 724
Youssef Ezzat Abdelaal	537 152 745
Pierre-Auguste Baziramwabo	537 291 293
Ayman El Hamri	537 033 330

Session d'hiver 2026
Table des matières

1-Diagramme de classes de conception (DCC)

2-Architecture logique

3-Diagrammes de séquence de conception (DSC)

3.1. Déterminer l'élément sur lequel a lieu un clic de souris.

3.1.1. Premier diagramme

3.1.2. Second diagramme.

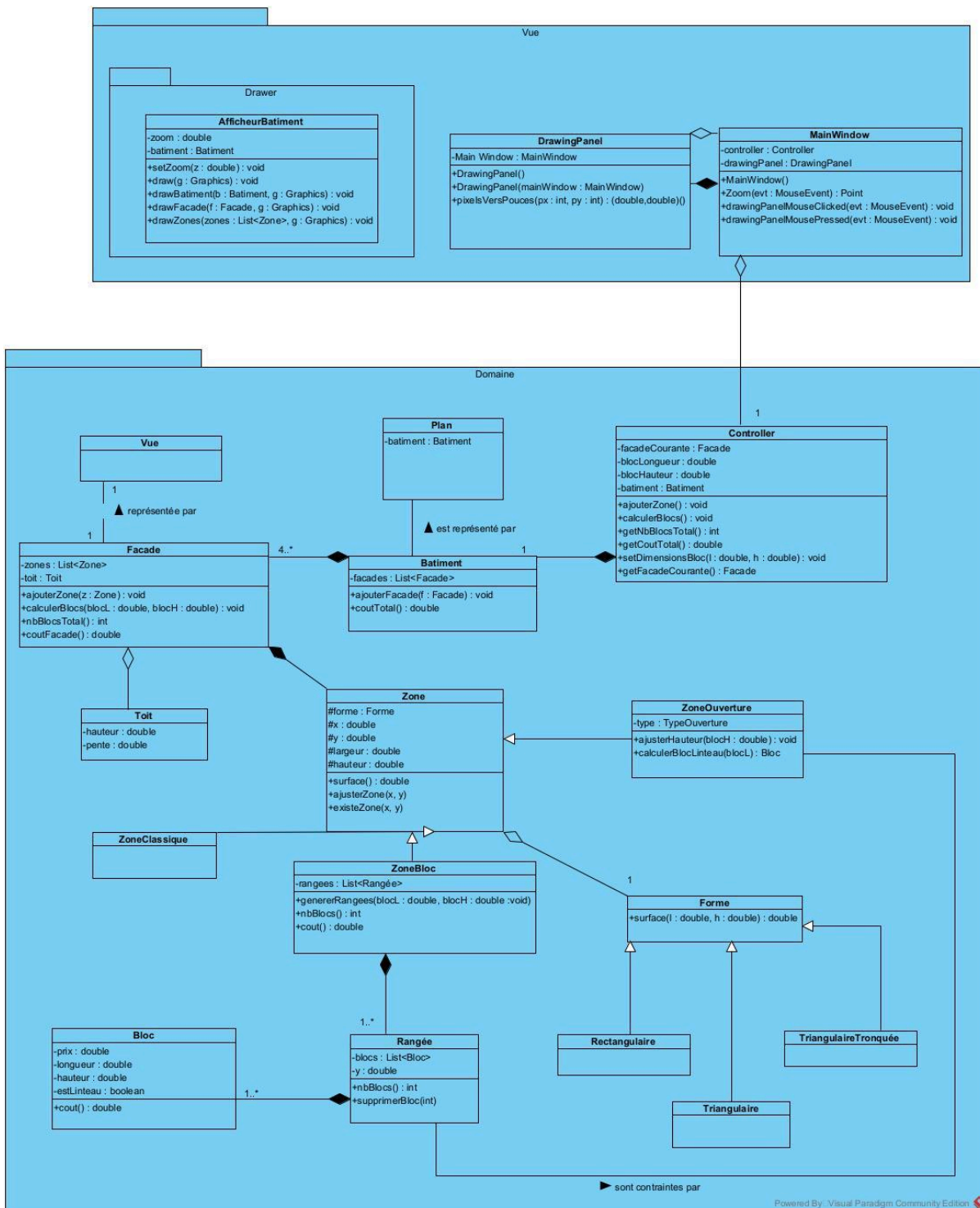
3.2. Ajout d'une zone.

3.3. Gestion de l'affichage de la vue.

4-Pseudocode – Simulation du placement des blocs

5-Diagramme de Gantt

1-Diagramme de classes de conception (DCC)



La classe `MainWindow` constitue le point central de l'interface graphique. Elle instancie les composants visuels, gère les événements utilisateur (clics, zoom) et délègue les traitements au `Controller`. Ce dernier joue le

rôle de contrôleur principal, conformément au principe du contrôleur de Larman, en assurant la coordination entre la couche Vue et le Domaine.

La classe `DrawingPanel` est responsable du rendu graphique. Elle capte les événements de la souris et délègue l'affichage à un objet spécialisé de type `AfficheurBatiment`, qui encapsule les opérations de dessin à l'aide de l'objet `Graphics`. Cette classe ne contient aucune logique métier et se limite au rendu visuel du bâtiment et de ses éléments.

Le package `Domaine` représente le cœur du système. La classe `Batiment` agit comme agrégat principal et contient une collection de `Facade`. Chaque façade regroupe des `Zone`, qui peuvent être de différents types (par exemple `ZoneClassique` ou `ZoneBloc`). Une `ZoneBloc` est composée de plusieurs `Rangée`, elles-mêmes constituées de plusieurs `Bloc`. Cette hiérarchie illustre clairement les relations de composition présentes dans le modèle.

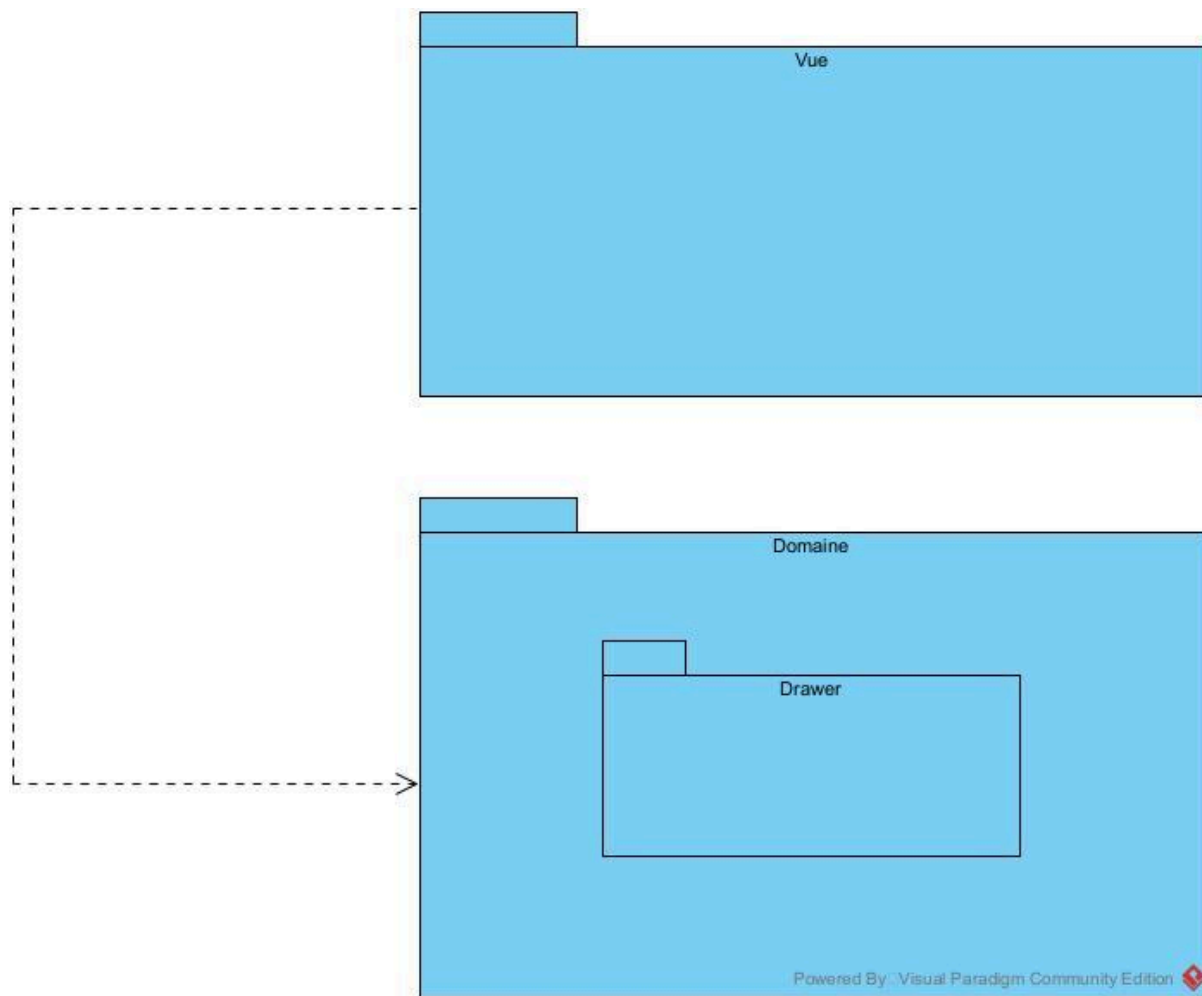
La classe `Forme` définit le calcul de surface et est spécialisée par différentes formes géométriques (`Rectangulaire`, `Triangulaire`, `TriangulaireTronquée`). Les dimensions et calculs sont encapsulés dans les classes du domaine, assurant ainsi que toute la logique métier (calcul du nombre de blocs, coût total, dimensions) soit centralisée dans cette couche.

La séparation des responsabilités est claire :

- La Vue gère l'affichage et les interactions graphiques ;
- Le Controller orchestre les opérations et invoque les méthodes du domaine ;
- Le Domaine contient l'état du système et applique les règles de calcul.

Cette organisation assure une architecture modulaire, cohérente et évolutive, facilitant l'ajout de nouveaux types de zones, de nouvelles formes ou de nouvelles règles de calcul sans impacter les couches d'affichage.

2-Architecture logique



L'architecture de l'application est divisée en deux principaux packages :
Vue et Domaine.

Vue

Le package *Vue* contient les classes responsables de l'interface graphique, comme *MainWindow* et *DrawingPanel*.

Il gère l'affichage des façades et des zones, ainsi que les interactions de l'utilisateur (clics, zoom).

Il délègue les traitements au Controller sans contenir de logique métier.

Domaine

Le package *Domaine* regroupe les classes représentant le modèle de données, comme *Batiment*, *Facade*, *Zone* et *Bloc*.

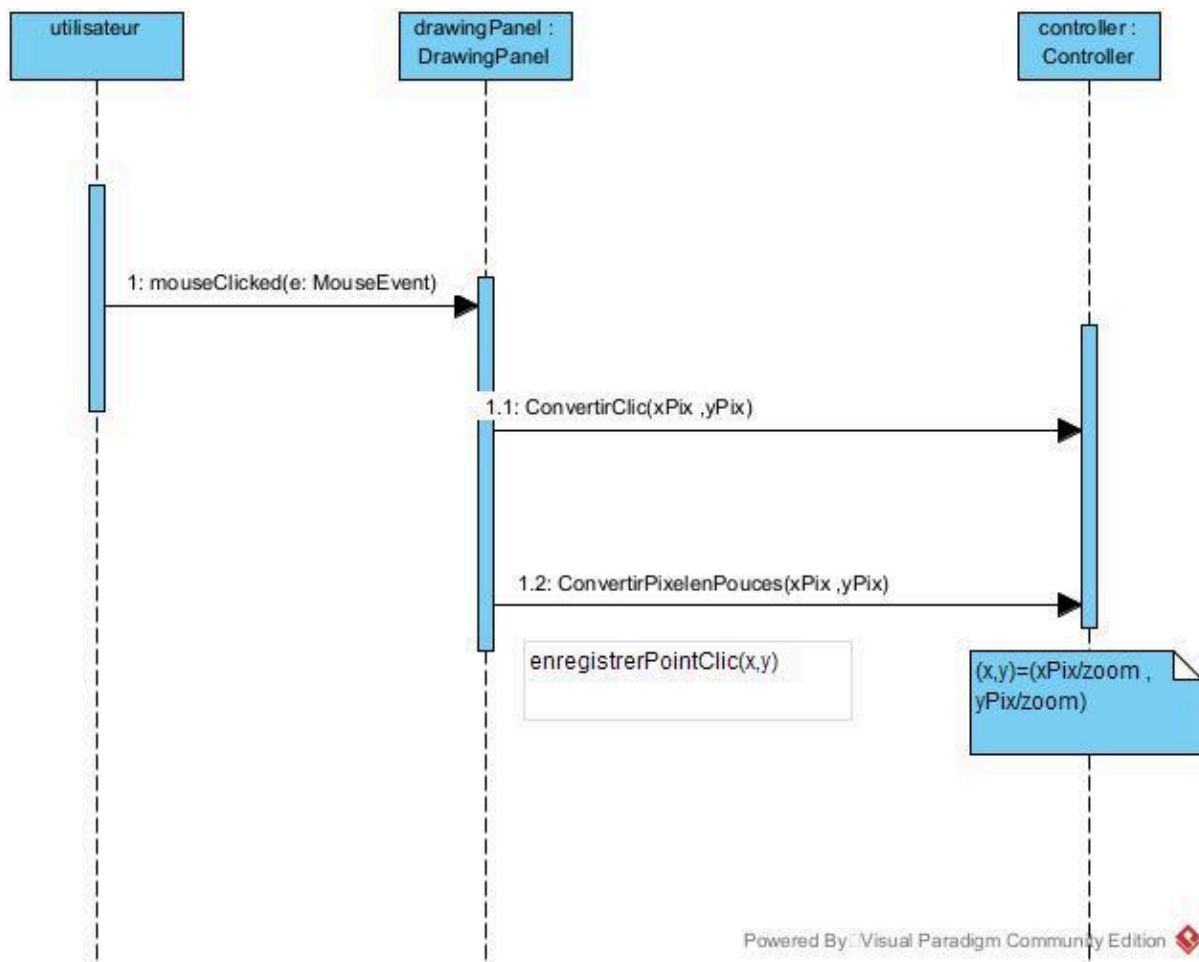
Il contient toute la logique métier, notamment les calculs du nombre de

blocs et du coût.

3-Diagrammes de séquence de conception (DSC)

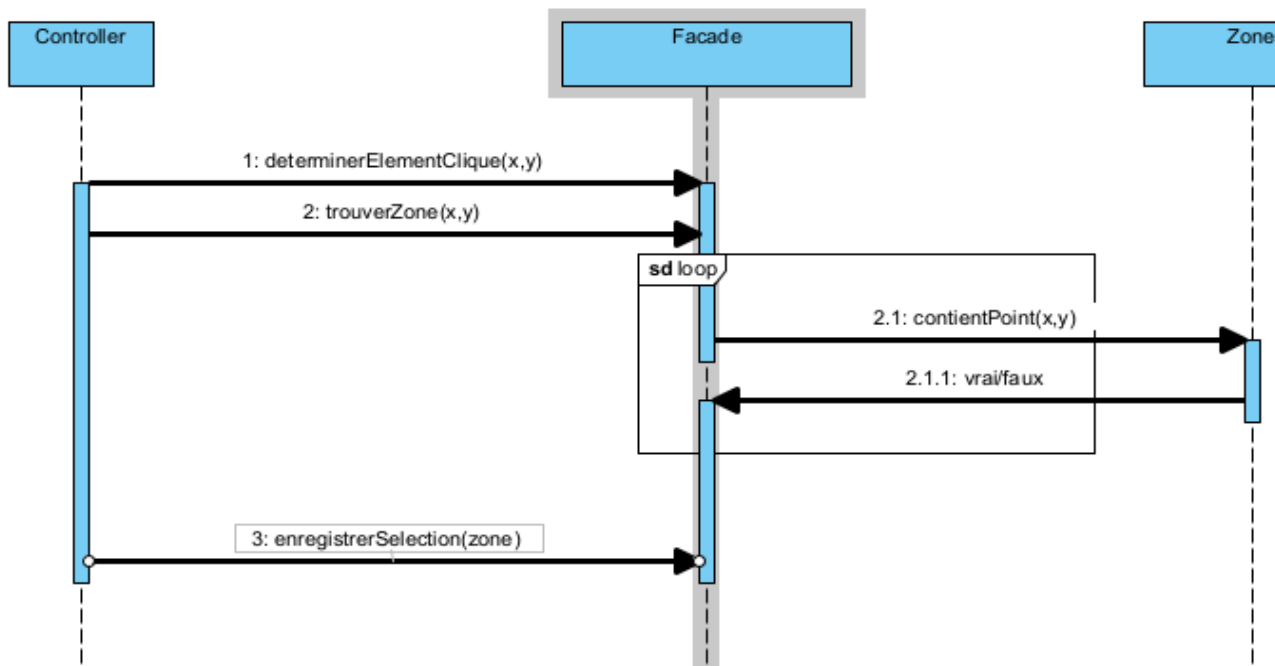
3.1. Déterminer l'élément sur lequel a lieu un clic de souris.

3.1.1. Premier diagramme



Ce diagramme illustre comment l'application récupère les coordonnées d'un clic de souris. Lorsque l'utilisateur clique dans la zone de dessin, l'événement est transmis au DrawingPanel, puis envoyé au Controller. Le Controller convertit ensuite les coordonnées du clic en pouces en tenant compte du niveau de zoom courant. Enfin, les coordonnées converties sont enregistrées afin de conserver la position exacte du clic, indépendamment du zoom appliqué.

3.1.2. Second diagramme.

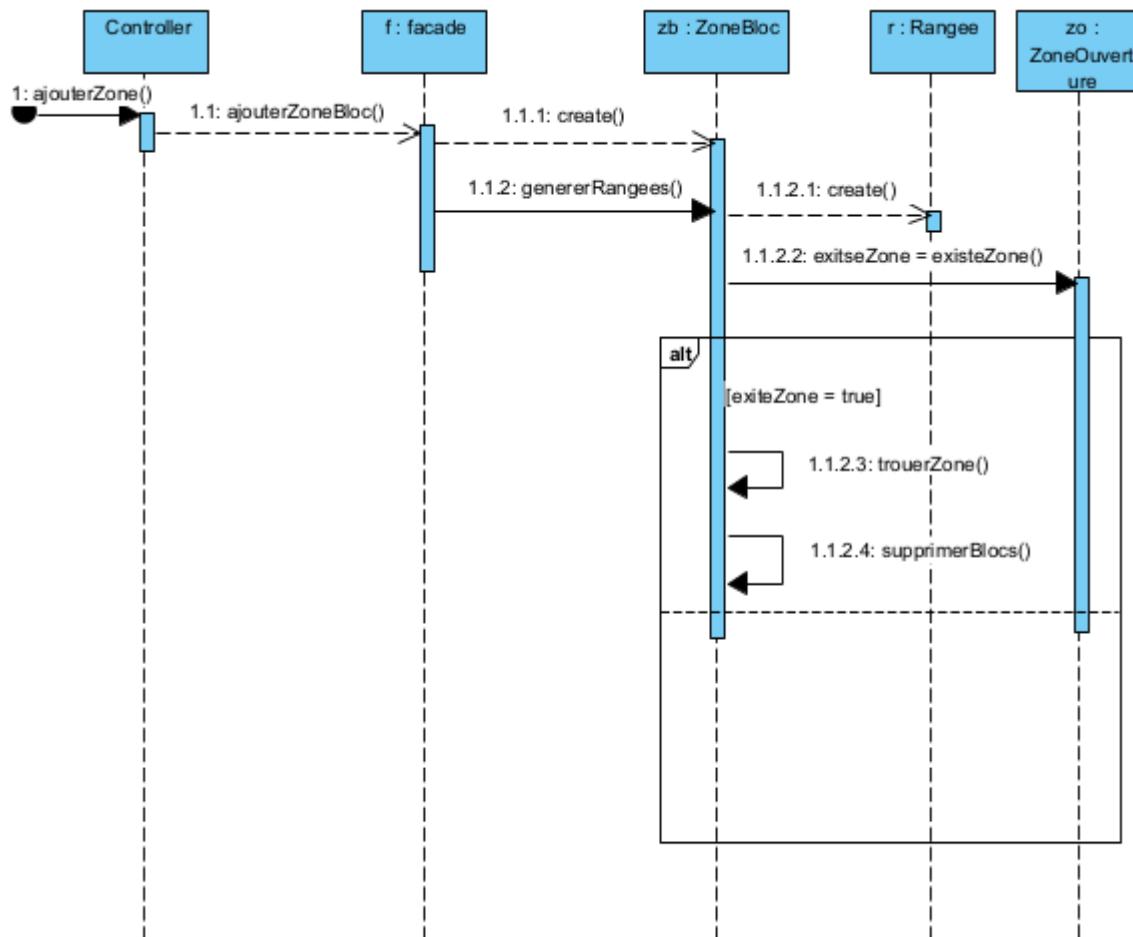


Ce diagramme illustre comment l'application détermine la zone sélectionnée après un clic. Le Controller reçoit les coordonnées du point (x, y) en pouces et demande à la façade de trouver la zone correspondante.

La façade vérifie chaque Zone à l'aide d'une boucle. Pour chaque zone, la méthode `contientPoint(x, y)` est appelée afin de vérifier si le point se trouve à l'intérieur de ses limites.

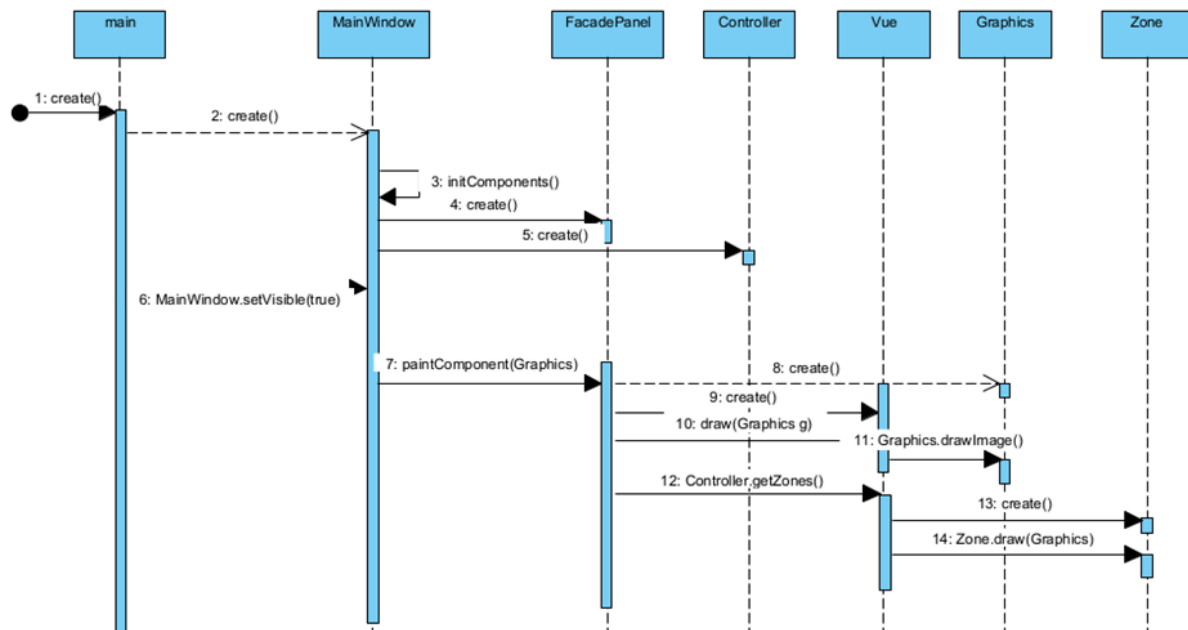
Si une zone contient le point, elle est retournée au Controller, qui enregistre alors la sélection. Cette étape permet d'identifier correctement l'élément sur lequel l'utilisateur a cliqué.

3.2. Ajout d'une zone.



A partir du controller la methode `ajouterZone()` est appele. Etant donne qu'on sous-entend que les parametres de la methode `ajouterZone()` sont bien mis pour l'ajout du zone bloc, la methode fait appel a la methode `ajouterZoneBloc()` qui provient de la classe facade. La methode `ajouterZoneBloc()` cree ensuite une instance de la classe `ZoneBloc`, qui, elle appelle la methode `genererRangees()`. La methode `genererRangees()` cree une instance de la classe `Rangee`, qui, elle, verifie s'il n'y a pas une zone d'ouverture qui encombre l'espace dans laquelle on veut inserer la zone de bloc. Si une ouverture prend une partie de l'espace, la rangee va se trouver avec la methode `trouerRangees()`, qui fait appelle a la methode `supprimerBlocs()`, pour laisser l'espace a la zone d'ouverture.

3.3 Affichage de la vue



Le programme de l'affichage commence avec de la méthode `main()`. Cette méthode crée une instance de `MainWindow` (la fenêtre principale). Lors de sa construction, `MainWindow` appelle `initComponents()`, qui permet d'initialiser les éléments graphiques. La classe `MainWindow` instancie ensuite un `FacadePanel` (zone d'affichage personnalisée) ainsi qu'un `ControllerClass`, puis ajoute le `FacadePanel` à la fenêtre avec `this.add(FacadePanel)`. Enfin, l'appel à `setVisible(true)` rend la fenêtre visible à l'écran.

Après, le système graphique Swing appelle automatiquement la méthode `paintComponent(Graphics)` du `FacadePanel` pour dessiner le contenu. Cette méthode commence par appeler `JPanel.paintComponent(Graphics)` afin d'assurer un affichage correct (rafraîchissement propre du panneau). Ensuite, le panneau déclenche le dessin de la vue via `Vue.draw(Graphics)`.

Pendant ce processus, FacadePanel demande au contrôleur les données des zones avec la methode `ControllerClass.getZones()`. Le contrôleur retourne alors les objets Zone, et pour chacun d'eux, la méthode `Zone.draw(Graphics)` est appelée afin qu'il se dessine lui-même.

Le diagramme et le texte explicatif sont inspirés d'explications ChatGPT.

4. Algorithme de simulation du placement et schémas explicatifs:

4.1. Approche algorithmique et contraintes:

Notre stratégie de simulation repose sur un algorithme de pavage par balayage (génération de grilles), optimisé pour respecter les contraintes structurelles de la maçonnerie. Le système utilise des blocs standards de 12" x 8".

Pour garantir la solidité du mur, l'algorithme intègre une règle d'emprunt (ou d'ajustement) : si la division de la largeur de la zone laisse une retaille inférieure à la taille minimale sécuritaire (6 pouces), l'algorithme fusionne cette retaille avec le bloc précédent pour redistribuer l'espace en deux blocs égaux et conformes.

4.2. Pseudocode de la simulation:

Voici la logique de base qui dicte le placement et l'ajustement des blocs :

```

ALGORITHME SimulationPlacement
ENTRÉES :
    zone_largeur, zone_hauteur
CONSTANTES :
    BLOC_L = 12, BLOC_H = 8, PRIX_UNITAIRE = 20, MIN_RETAILLE = 6
SORTIES :
    nb_total_blocs, cout_total, liste_coordonnees

DÉBUT
    // 1. Calcul du nombre de rangées (Hauteur)
    nb_rangees <- ENTIER(zone_hauteur / BLOC_H)

    // 2. Analyse de la largeur et de la retaille (Modulo)
    nb_colonnes_pleines <- ENTIER(zone_largeur / BLOC_L)
    retaille <- zone_largeur MODULO BLOC_L

    // 3. Application de la Règle d'emprunt
    SI (retaille > 0) ET (retaille < MIN_RETAILLE) ALORS
        // On emprunte au bloc précédent : (12 + retaille) / 2
        largeur_bloc_ajuste <- (BLOC_L + retaille) / 2
        nb_colonnes_pleines <- nb_colonnes_pleines - 1
        nb_blocs_coupes <- 2
    SINON SI (retaille >= MIN_RETAILLE) ALORS
        nb_blocs_coupes <- 1
    SINON
        nb_blocs_coupes <- 0
    FIN SI

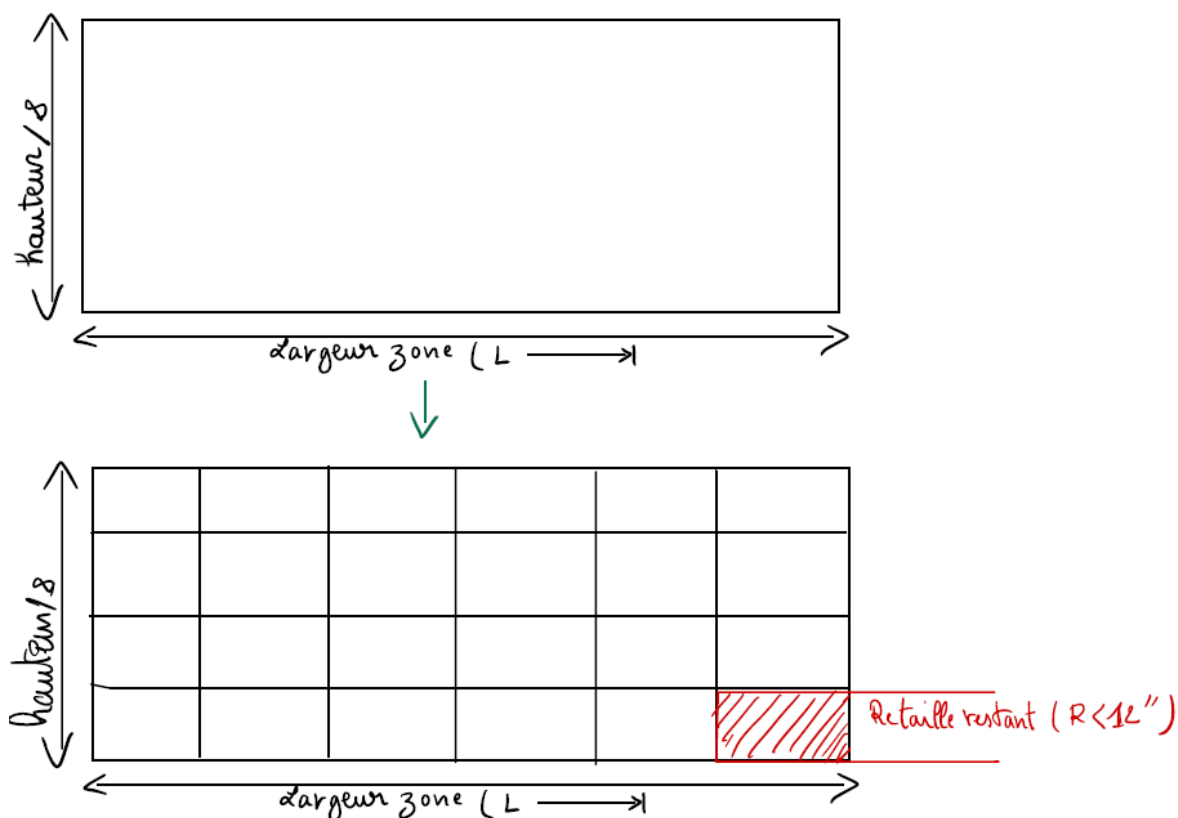
    // 4. Bilan et Estimation
    blocs_par_rangee <- nb_colonnes_pleines + nb_blocs_coupes
    nb_total_blocs <- blocs_par_rangee * nb_rangees
    cout_total <- nb_total_blocs * PRIX_UNITAIRE

    RETOURNER nb_total_blocs, cout_total
FIN

```

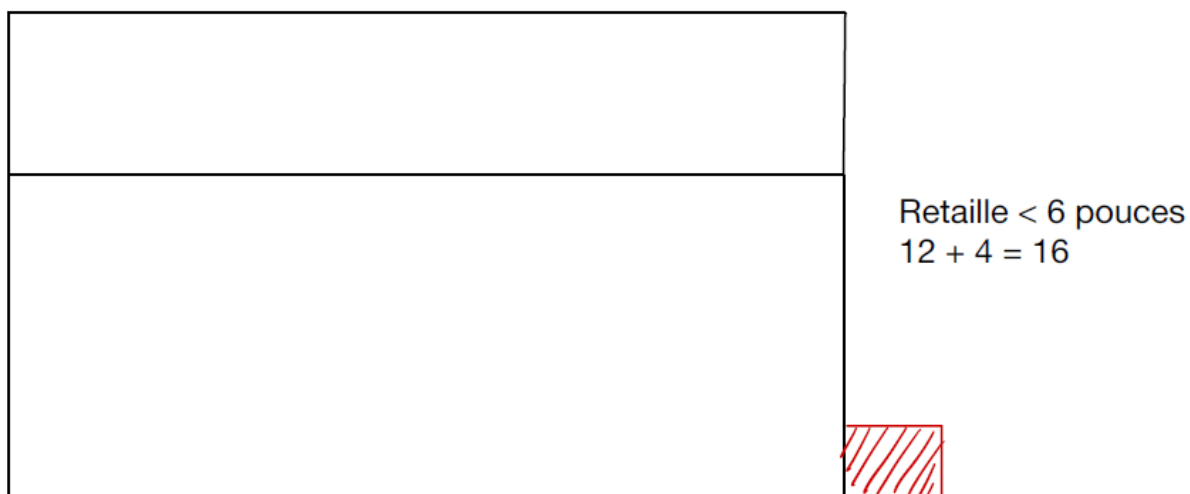
4.3. Schémas explicatifs du placement:

Etape 1: Definition de la geometrie



Étape 1 : Définition de la géométrie. > Le système reçoit les dimensions de la zone (Largeur L et Hauteur H). Une grille théorique est projetée pour évaluer la capacité maximale en blocs de 12" x 8".

Etape 2 : Application de la regle d'ajustement





Retaille > 6 pouces
 $8 + 8 = 16$

Étape 2 : Application de la règle d'ajustement. > L'algorithme détecte une anomalie structurelle : la retaille en fin de rangée est inférieure à 6 pouces (ex: 4"). Pour éviter ce fragment fragile, le système emprunte le bloc précédent (12"). L'espace total de 16" est alors divisé en deux blocs robustes de 8", respectant ainsi la contrainte.

Etape 3 : Résultat final et estimation



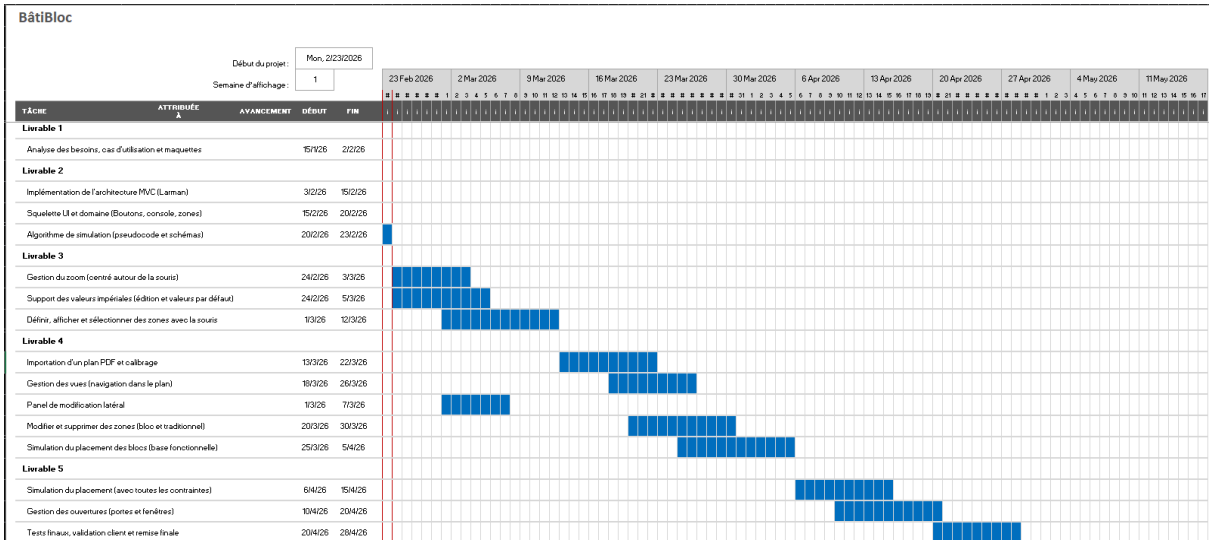
Le système génère le bilan final pour l'utilisateur, incluant le nombre total de blocs consommés et le coût total estimé.

-Nombre total : 24 blocs

-Coût estimé : 480 \$

Étape 3 : Résultat final et estimation. > La zone est entièrement pavée.
Le système génère le bilan final pour l'utilisateur, incluant le nombre total de blocs consommés (entiers et retailés) et le coût total estimé, confirmant que toutes les retailles minimales sont respectées.

5. Gantt :



Contribution de l'équipe

Nom	Tâche
Youssef Dhaoui	1 / 2 /3.1 /3.2
Abdallah Chenah	1 /2 /3.1 /3.2
Youssef Ezzat Abdelaal	4./5 /7
Pierre-Auguste Baziramwabo	3.2 /3.3
Ayman El Hamri	1 /2

